

Teori Artificial Neural Network

Ryan Fadhilah Faizal Hakim¹

¹Sains Data, Fakultas Sains dan Teknologi, IKOPIN University

1 Pendahuluan

Artificial Neural Network (ANN), atau jaringan saraf tiruan, adalah model komputasi yang terinspirasi dari cara kerja sistem saraf biologis. ANN terdiri atas sekumpulan unit komputasi sederhana yang disebut neuron atau node. Neuron-neuron tersebut tersusun dalam lapisan-lapisan dan saling terhubung melalui bobot yang dapat dipelajari dari data.

ANN digunakan untuk mempelajari hubungan kompleks antara input dan output. Jika tersedia pasangan data

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N, \quad (1)$$

dengan $\mathbf{x}^{(i)} \in \mathbb{R}^d$ sebagai input dan $\mathbf{y}^{(i)}$ sebagai target, maka tujuan ANN adalah mencari fungsi aproksimasi

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad (2)$$

yang memetakan input ke output sedekat mungkin dengan target sebenarnya. Parameter θ mencakup seluruh bobot dan bias dalam jaringan.

ANN merupakan dasar dari berbagai arsitektur *deep learning*, termasuk convolutional neural network, recurrent neural network, autoencoder, generative adversarial network, dan transformer. Model ini dapat digunakan untuk klasifikasi, regresi, prediksi deret waktu, pengenalan pola, pemrosesan bahasa alami, visi komputer, sistem rekomendasi, dan pengambilan keputusan.

2 Neuron Buatan

Unit dasar ANN adalah neuron buatan. Sebuah neuron menerima input $x_1, x_2, \dots, x_d$, mengalikan setiap input dengan bobot $w_1, w_2, \dots, w_d$, menambahkan bias b , lalu menerapkan fungsi aktivasi. Secara matematis, nilai pra-aktivasi neuron adalah

$$z = \sum_{j=1}^d w_j x_j + b. \quad (3)$$

Dalam notasi vektor,

$$z = \mathbf{w}^T \mathbf{x} + b, \quad (4)$$

dengan $\mathbf{w} \in \mathbb{R}^d$ dan $\mathbf{x} \in \mathbb{R}^d$. Output neuron adalah

$$a = \phi(z) = \phi(\mathbf{w}^T \mathbf{x} + b), \quad (5)$$

di mana $\phi(\cdot)$ adalah fungsi aktivasi.

Jika tidak ada fungsi aktivasi nonlinier, maka gabungan banyak lapisan linear tetap ekuivalen dengan satu transformasi linear. Oleh karena itu, fungsi aktivasi sangat penting agar ANN mampu memodelkan hubungan nonlinier.

3 Perceptron

Perceptron adalah bentuk paling sederhana dari jaringan saraf. Untuk klasifikasi biner, perceptron menghasilkan prediksi

$$\hat{y} = \begin{cases} 1, & \mathbf{w}^T \mathbf{x} + b \geq 0, \\ 0, & \mathbf{w}^T \mathbf{x} + b < 0. \end{cases} \quad (6)$$

Batas keputusan perceptron adalah hiperbidang

$$\mathbf{w}^T \mathbf{x} + b = 0. \quad (7)$$

Untuk data dua dimensi, batas keputusan ini berupa garis. Untuk data berdimensi lebih tinggi, batas tersebut berupa hiperbidang.

Aturan pembaruan perceptron untuk sampel ke- i dapat ditulis sebagai

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y^{(i)} - \hat{y}^{(i)})\mathbf{x}^{(i)}, \quad (8)$$

$$b \leftarrow b + \eta(y^{(i)} - \hat{y}^{(i)}), \quad (9)$$

dengan η adalah learning rate. Perceptron hanya dapat menyelesaikan masalah yang terpisah secara linear, sehingga dibutuhkan jaringan multilayer untuk masalah yang lebih kompleks.

4 Arsitektur Multilayer Perceptron

Multilayer Perceptron (MLP) adalah ANN feedforward yang memiliki satu atau lebih hidden layer. Misalkan jaringan memiliki L lapisan. Input dinotasikan sebagai

$$\mathbf{a}^{(0)} = \mathbf{x}. \quad (10)$$

Untuk lapisan ke- ℓ , transformasi linear dan aktivasi diberikan oleh

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad (11)$$

$$\mathbf{a}^{(\ell)} = \phi^{(\ell)}(\mathbf{z}^{(\ell)}), \quad \ell = 1, 2, \dots, L. \quad (12)$$

Di sini,

- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ adalah matriks bobot lapisan ke- ℓ ,
- $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ adalah vektor bias,
- $\mathbf{a}^{(\ell)} \in \mathbb{R}^{n_\ell}$ adalah aktivasi lapisan ke- ℓ ,
- n_ℓ adalah jumlah neuron pada lapisan ke- ℓ .

Output jaringan adalah

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)} = f_\theta(\mathbf{x}). \quad (13)$$

Secara komposisional, MLP dapat ditulis sebagai

$$f_\theta(\mathbf{x}) = f_L \circ f_{L-1} \circ \dots \circ f_1(\mathbf{x}). \quad (14)$$

Hal ini menunjukkan bahwa ANN membangun fungsi kompleks melalui komposisi transformasi linear dan nonlinier.

5 Forward Propagation

Forward propagation adalah proses menghitung output jaringan dari input. Untuk satu sampel, prosesnya dimulai dari $\mathbf{a}^{(0)} = \mathbf{x}$, lalu menghitung $\mathbf{z}^{(1)}, \mathbf{a}^{(1)}, \dots, \mathbf{z}^{(L)}, \mathbf{a}^{(L)}$.

Untuk mini-batch berisi B sampel, input dapat ditulis sebagai matriks

$$\mathbf{A}^{(0)} = \mathbf{X} \in \mathbb{R}^{n_0 \times B}. \quad (15)$$

Forward propagation pada mini-batch adalah

$$\mathbf{Z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{A}^{(\ell-1)} + \mathbf{b}^{(\ell)} \mathbf{1}^T, \quad (16)$$

$$\mathbf{A}^{(\ell)} = \phi^{(\ell)}(\mathbf{Z}^{(\ell)}), \quad (17)$$

dengan $\mathbf{1} \in \mathbb{R}^B$ adalah vektor satu. Bias ditambahkan ke setiap kolom mini-batch.

6 Fungsi Aktivasi

Fungsi aktivasi menentukan sifat nonlinier jaringan. Beberapa fungsi aktivasi penting adalah sebagai berikut.

6.1 Sigmoid

Fungsi sigmoid didefinisikan sebagai

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (18)$$

Turunannya adalah

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (19)$$

Sigmoid menghasilkan nilai pada interval $(0, 1)$ sehingga sering digunakan untuk probabilitas pada klasifikasi biner. Namun, sigmoid dapat mengalami saturasi ketika z sangat besar atau sangat kecil, yang menyebabkan gradien menjadi kecil.

6.2 Tangent Hyperbolic

Fungsi tanh didefinisikan sebagai

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (20)$$

Turunannya adalah

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z). \quad (21)$$

Fungsi ini menghasilkan nilai pada interval $(-1, 1)$ dan memiliki keluaran terpusat di sekitar nol.

6.3 ReLU

ReLU atau *Rectified Linear Unit* adalah

$$\text{ReLU}(z) = \max(0, z). \quad (22)$$

Turunannya adalah

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (23)$$

ReLU banyak digunakan karena sederhana, efisien, dan membantu mengurangi masalah vanishing gradient.

6.4 Leaky ReLU dan Softmax

Leaky ReLU didefinisikan sebagai

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad 0 < \alpha < 1. \quad (24)$$

Fungsi ini mengurangi risiko neuron mati pada ReLU.

Untuk klasifikasi multi-kelas, fungsi softmax digunakan pada lapisan output:

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, 2, \dots, K. \quad (25)$$

Softmax menghasilkan distribusi probabilitas karena

$$0 < \hat{y}_k < 1, \quad \sum_{k=1}^K \hat{y}_k = 1. \quad (26)$$

7 Fungsi Loss

Fungsi loss mengukur seberapa jauh prediksi model dari target sebenarnya. Untuk regresi, loss yang umum adalah mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \left\| \mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)} \right\|_2^2. \quad (27)$$

Untuk klasifikasi biner, digunakan binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]. \quad (28)$$

Untuk klasifikasi multi-kelas dengan label one-hot $\mathbf{y}^{(i)}$, digunakan categorical cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}). \quad (29)$$

Jika label benar untuk sampel ke- i adalah kelas c_i , maka

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c_i}^{(i)}). \quad (30)$$

Tujuan pelatihan adalah mencari parameter yang meminimalkan fungsi objektif

$$\theta^* = \arg \min_{\theta} \mathcal{J}(\theta), \quad (31)$$

dengan

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}). \quad (32)$$

8 Backpropagation

Backpropagation adalah algoritma untuk menghitung gradien fungsi loss terhadap seluruh parameter jaringan secara efisien menggunakan aturan rantai. Untuk jaringan MLP, misalkan

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad (33)$$

$$\mathbf{a}^{(\ell)} = \phi^{(\ell)}(\mathbf{z}^{(\ell)}). \quad (34)$$

Definisikan error lokal pada lapisan ke- ℓ sebagai

$$\boldsymbol{\delta}^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}. \quad (35)$$

Untuk lapisan output,

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi' \left(\mathbf{z}^{(L)} \right), \quad (36)$$

dengan $\odot$ menyatakan perkalian elemen per elemen. Untuk hidden layer,

$$\boldsymbol{\delta}^{(\ell)} = \left((\mathbf{W}^{(\ell+1)})^T \boldsymbol{\delta}^{(\ell+1)} \right) \odot \phi' \left(\mathbf{z}^{(\ell)} \right). \quad (37)$$

Gradien terhadap bobot dan bias adalah

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} (\mathbf{a}^{(\ell-1)})^T, \quad (38)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}. \quad (39)$$

Untuk mini-batch berukuran B , gradien bobot menjadi

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(\ell)}} = \frac{1}{B} \boldsymbol{\Delta}^{(\ell)} (\mathbf{A}^{(\ell-1)})^T, \quad (40)$$

dan gradien bias adalah

$$\frac{\partial \mathcal{J}}{\partial \mathbf{b}^{(\ell)}} = \frac{1}{B} \boldsymbol{\Delta}^{(\ell)} \mathbf{1}, \quad (41)$$

dengan $\boldsymbol{\Delta}^{(\ell)}$ berisi error lokal semua sampel dalam mini-batch.

9 Softmax dan Cross-Entropy

Kombinasi softmax dan cross-entropy memiliki bentuk gradien yang sederhana. Misalkan logit output adalah $\mathbf{z}$ dan prediksi softmax adalah

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}. \quad (42)$$

Loss cross-entropy untuk satu sampel adalah

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{y}_k). \quad (43)$$

Maka gradien loss terhadap logit adalah

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k. \quad (44)$$

Persamaan ini sangat penting karena menyederhanakan perhitungan backpropagation pada lapisan output klasifikasi multi-kelas.

10 Optimisasi Parameter

Setelah gradien dihitung, parameter diperbarui menggunakan algoritma optimisasi. Pada gradient descent, pembaruan parameter adalah

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{J}(\theta_t), \quad (45)$$

dengan η adalah learning rate.

Pada stochastic gradient descent (SGD), gradien dihitung dari satu sampel atau mini-batch, bukan seluruh dataset. Jika mini-batch pada iterasi ke- t adalah $\mathcal{B}_t$, maka

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \mathcal{L}^{(i)}(\theta_t). \quad (46)$$

Dengan momentum, pembaruan menjadi

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t - \eta \nabla_{\theta} \mathcal{J}(\theta_t), \quad (47)$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1}, \quad (48)$$

dengan μ adalah koefisien momentum.

Adam menggunakan estimasi momen pertama dan kedua dari gradien:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (49)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (50)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (51)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (52)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (53)$$

11 Inisialisasi Bobot

Inisialisasi bobot sangat memengaruhi stabilitas pelatihan. Jika bobot terlalu kecil, sinyal dan gradien dapat menghilang. Jika terlalu besar, aktivasi dan gradien dapat meledak. Inisialisasi Xavier/Glorot cocok untuk aktivasi sigmoid atau tanh. Untuk lapisan dengan n_{in} input dan n_{out} output, bobot dapat diinisialisasi dari distribusi uniform

$$W_{ij} \sim U \left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right). \quad (54)$$

Untuk aktivasi ReLU, inisialisasi He sering digunakan:

$$W_{ij} \sim \mathcal{N} \left(0, \frac{2}{n_{in}} \right). \quad (55)$$

Tujuannya adalah menjaga varians aktivasi dan gradien tetap stabil sepanjang lapisan.

12 Vanishing dan Exploding Gradient

Pada jaringan yang sangat dalam, gradien dapat mengecil atau membesar secara eksponensial. Misalkan untuk jaringan sederhana gradien antar lapisan melibatkan produk Jacobian:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \prod_{r=\ell+1}^L \frac{\partial \mathbf{a}^{(r)}}{\partial \mathbf{a}^{(r-1)}}. \quad (56)$$

Jika norma Jacobian rata-rata kurang dari satu, gradien cenderung menghilang:

$$\left\| \prod_{r=\ell+1}^L \mathbf{J}^{(r)} \right\| \rightarrow 0. \quad (57)$$

Jika norma Jacobian rata-rata lebih besar dari satu, gradien dapat meledak:

$$\left\| \prod_{r=\ell+1}^L \mathbf{J}^{(r)} \right\| \rightarrow \infty. \quad (58)$$

Solusi umum meliputi penggunaan ReLU, inisialisasi bobot yang baik, batch normalization, residual connection, gradient clipping, dan pemilihan learning rate yang tepat.

13 Regularisasi

ANN dengan banyak parameter rentan mengalami overfitting. Regularisasi digunakan untuk meningkatkan kemampuan generalisasi.

13.1 Regularisasi L2

Regularisasi L2 menambahkan penalti terhadap norma bobot:

$$\mathcal{J}_{\text{reg}}(\theta) = \mathcal{J}(\theta) + \lambda \sum_{\ell=1}^L \left\| \mathbf{W}^{(\ell)} \right\|_F^2, \quad (59)$$

dengan λ adalah koefisien regularisasi dan $\|\cdot\|_F$ adalah norma Frobenius. Gradiennya menjadi

$$\frac{\partial \mathcal{J}_{\text{reg}}}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(\ell)}} + 2\lambda \mathbf{W}^{(\ell)}. \quad (60)$$

13.2 Dropout

Dropout menonaktifkan sebagian neuron secara acak selama pelatihan. Jika $\mathbf{a}$ adalah aktivasi dan $\mathbf{m}$ adalah mask Bernoulli, maka

$$\tilde{\mathbf{a}} = \frac{\mathbf{m} \odot \mathbf{a}}{q}, \quad m_i \sim \text{Bernoulli}(q), \quad (61)$$

dengan q adalah probabilitas neuron dipertahankan. Faktor $1/q$ menjaga ekspektasi aktivasi tetap stabil.

13.3 Early Stopping

Early stopping menghentikan pelatihan ketika loss validasi tidak lagi membaik. Teknik ini dapat dipandang sebagai regularisasi karena mencegah model terlalu menyesuaikan diri terhadap data pelatihan.

14 Normalisasi

Batch normalization menstabilkan distribusi aktivasi selama pelatihan. Untuk mini-batch berisi B nilai aktivasi, yaitu $x_1, x_2, \dots, x_B$, batch normalization menghitung

$$\mu_B = \frac{1}{B} \sum_{i=1}^B x_i, \quad (62)$$

$$\sigma_B^2 = \frac{1}{B} \sum_{i=1}^B (x_i - \mu_B)^2, \quad (63)$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (64)$$

$$y_i = \gamma \hat{x}_i + \beta. \quad (65)$$

Parameter γ dan β dipelajari agar jaringan dapat mengatur kembali skala dan pergeseran aktivasi. Batch normalization sering mempercepat konvergensi dan membuat pelatihan lebih stabil.

15 Universal Approximation Theorem

Salah satu dasar teoritis ANN adalah *Universal Approximation Theorem*. Secara informal, teorema ini menyatakan bahwa jaringan feedforward dengan satu hidden layer yang memiliki jumlah neuron cukup banyak dan fungsi aktivasi nonlinier tertentu dapat mengaproksimasi fungsi kontinu pada domain kompak dengan tingkat ketelitian sebarang.

Secara matematis, untuk fungsi kontinu f pada himpunan kompak $K \subset \mathbb{R}^d$ dan untuk setiap $\epsilon > 0$, terdapat jaringan

$$g(\mathbf{x}) = \sum_{j=1}^M \alpha_j \phi(\mathbf{w}_j^T \mathbf{x} + b_j) \quad (66)$$

sehingga

$$\sup_{\mathbf{x} \in K} |f(\mathbf{x}) - g(\mathbf{x})| < \epsilon. \quad (67)$$

Teorema ini menunjukkan bahwa ANN memiliki kapasitas representasi yang sangat kuat. Namun, teorema tersebut tidak menjamin bahwa proses pelatihan akan selalu menemukan parameter terbaik secara mudah.

16 Bias-Variance Tradeoff

Kemampuan generalisasi model dipengaruhi oleh bias dan varians. Model yang terlalu sederhana memiliki bias tinggi dan cenderung underfitting. Model yang terlalu kompleks memiliki varians tinggi dan cenderung overfitting.

Secara konseptual, error prediksi dapat diuraikan sebagai

$$\mathbb{E} \left[(y - \hat{f}(\mathbf{x}))^2 \right] = \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}) + \sigma^2, \quad (68)$$

dengan σ^2 adalah noise irreducible. Tujuan pelatihan adalah mencapai keseimbangan antara bias dan varians agar model bekerja baik pada data baru.

17 Evaluasi Model

Untuk klasifikasi, metrik evaluasi umum meliputi akurasi, presisi, recall, dan F1-score. Akurasi didefinisikan sebagai

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (69)$$

Presisi adalah

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (70)$$

recall adalah

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (71)$$

dan F1-score adalah

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (72)$$

Untuk regresi, metrik umum meliputi mean absolute error (MAE), mean squared error (MSE), dan root mean squared error (RMSE):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}|, \quad (73)$$

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2, \quad (74)$$

$$\text{RMSE} = \sqrt{\text{MSE}}. \quad (75)$$

18 Contoh Perhitungan Sederhana

Misalkan sebuah neuron memiliki input $\mathbf{x} = [2, -1]^T$, bobot $\mathbf{w} = [0.5, -0.25]^T$, dan bias $b = 0.1$. Nilai pra-aktivasi adalah

$$z = \mathbf{w}^T \mathbf{x} + b = (0.5)(2) + (-0.25)(-1) + 0.1 = 1.35. \quad (76)$$

Jika fungsi aktivasi adalah sigmoid, maka output neuron adalah

$$a = \sigma(1.35) = \frac{1}{1 + e^{-1.35}} \approx 0.794. \quad (77)$$

Jika target biner adalah $y = 1$, maka binary cross-entropy untuk sampel ini adalah

$$\mathcal{L} = -\log(0.794) \approx 0.230. \quad (78)$$

19 Kelebihan dan Keterbatasan ANN

Kelebihan ANN meliputi:

- mampu memodelkan hubungan nonlinier yang kompleks,
- dapat digunakan untuk klasifikasi, regresi, dan representasi fitur,
- fleksibel dan dapat dikembangkan menjadi berbagai arsitektur deep learning,
- mampu belajar langsung dari data tanpa aturan eksplisit,
- dapat bekerja pada data berdimensi tinggi jika dirancang dengan baik.

Keterbatasan ANN meliputi:

- membutuhkan data yang cukup banyak untuk generalisasi yang baik,
- pelatihan dapat mahal secara komputasi,
- interpretabilitas sering lebih rendah dibanding model sederhana,
- sensitif terhadap inisialisasi, learning rate, dan hiperparameter,
- rentan terhadap overfitting jika tidak diberi regularisasi.

20 Perbandingan Model Linear dan ANN

Aspek	Model Linear	Artificial Neural Network
Bentuk fungsi	Linear terhadap fitur input	Nonlinier melalui aktivasi dan banyak lapisan
Kapasitas representasi	Terbatas	Sangat fleksibel
Interpretabilitas	Relatif tinggi	Lebih sulit dijelaskan
Kebutuhan data	Lebih sedikit	Biasanya lebih banyak
Komputasi	Lebih ringan	Lebih mahal
Aplikasi	Masalah sederhana dan baseline	Pola kompleks, data besar, deep learning

21 Aplikasi ANN

ANN digunakan dalam berbagai bidang, antara lain:

1. **Klasifikasi:** pengenalan digit, diagnosis penyakit, deteksi spam, dan klasifikasi sentimen.
2. **Regresi:** prediksi harga rumah, prediksi permintaan, dan estimasi risiko.
3. **Visi komputer:** pengenalan objek, segmentasi citra, dan deteksi wajah.
4. **Pemrosesan bahasa alami:** klasifikasi teks, penerjemahan, dan analisis sentimen.
5. **Sistem rekomendasi:** rekomendasi produk, film, musik, dan konten digital.
6. **Keuangan:** deteksi fraud, penilaian kredit, dan prediksi pasar.
7. **Kesehatan:** prediksi penyakit, analisis citra medis, dan personalisasi pengobatan.

22 Kesimpulan

Artificial Neural Network adalah model pembelajaran mesin yang mempelajari fungsi kompleks melalui komposisi transformasi linear dan fungsi aktivasi nonlinier. Struktur dasarnya terdiri atas neuron, bobot, bias, fungsi aktivasi, lapisan tersembunyi, fungsi loss, dan algoritma optimisasi. Secara matematis, ANN melakukan forward propagation untuk menghasilkan prediksi dan back-propagation untuk menghitung gradien terhadap parameter.

Kekuatan utama ANN adalah kemampuannya mengaproksimasi fungsi nonlinier yang kompleks, sebagaimana didukung oleh Universal Approximation Theorem. Namun, keberhasilan ANN dalam praktik sangat bergantung pada desain arsitektur, kualitas data, inisialisasi bobot, fungsi aktivasi, regularisasi, normalisasi, optimizer, dan pemilihan hiperparameter. Dengan dasar-dasar ini, ANN menjadi fondasi utama bagi banyak model deep learning modern.